

CSE234 Week 8: For Each and Searching Algorithms

This week we will look at the `std::for_each` function and the searching algorithms provided by the STL Algorithms library.

`std::for_each` is used to run a callback on each element in a range. The searching functions do not modify the elements of a container. They are used to check if an element is (or isn't) a member of a container, find if a container contains an element (or range of elements), and count the occurrences of an element in a container.

For Each

The first algorithm we will look at is `std::for_each`. This function has three parameters: the starting iterator of a range, the ending iterator of a range, and a unary function that takes an element of the collection as a parameter. `std::for_each` is used when the elements of a container (or consecutive subset of the container) need to be accessed, and possibly modified. `std::for_each` cannot be used to add or remove elements from the range, but can modify the existing elements.

```
// must include algorithm
#include <algorithm>
#include <iostream>
#include <array>

int main() {
    std::array<int, 7> values = { 2, 6, 24, 76, 3, 67, 5 };
    int sum = 0;

    // use for_each to sum elements
    std::for_each(values.begin(), values.end(), [&sum](int x) mutable)
        sum += x;
};

// compute average
double average = sum / 7.0;
std::cout << "Average = " << average << std::endl;
```

```
    return 0;  
}
```

The difference between `std::for_each` and a range-based for loop (`for (auto& x: y) {}` syntax) is that `std::for_each` supports running on a partial range, and, if the container supports reverse iterators, iterate backwards over the range. However, range-based for loops support `break` and `continue` statements, which `std::for_each` does not.

Any, All, and None

The first three searching algorithms we'll discuss return `bool`. The first function, `std::any_of`, returns true if **at least one** element in the collection meet the search criteria. `std::all_of` returns true if **all** items in the collection meet the search criteria. Finally, `std::none_of` returns true if **no** elements meet the criteria. All three functions have three arguments: the starting iterator, the ending iterator, and a unary predicate. Below are some examples of each function's usage:

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values = { 1, -2, 5, 34, 2, 90, 365, -67 };

    // this will return true since there is
    // at least one negative value in the vector
    if (std::any_of(values.begin(), values.end(), [](int x) { return x
        std::cout << "values contains a negative number" << std::endl;
    else
        std::cout << "values contains no negatives" << std::endl;

    // this will be false since there are even numbers in values
    if (std::all_of(vales.begin(), values.end(), [](int x) { return x
        std::cout << "All values are odd" << std::endl;
    else
        std::cout << "Some values are even" << std::endl;

    // this will be true since there are no 0s in values
    if (std::none_of(values.begin(), values.end(), [](int x) { return
        std::cout << "There are no 0s in values" << std::endl;
    else
        std::cout << "There is at least one 0 in values" << std::endl;

    return 0;
}

```

To be more concrete with our definitions, `any_of` returns true iff (if and only if) the predicate returns true at least once. `all_of` returns true iff the predicate returns false zero times. `none_of` returns true iff the predicate returns true zero times.

Find, Find If, and Find If Not

If you want to find a certain element, or if a certain element matches (or doesn't match) a criterion, then `std::find`, `std::find_if`, and `std::find_if_not` can be used. `std::find` takes a starting and ending iterator as its first two parameters, and the value to search for as its third parameter. If you are not looking for a specific value, but if a value in the collection meets a certain condition, `std::find_if` can be used. `find_if` also takes a start and end iterator as its first two parameters, but instead of a value it expects a predicate as the third argument. If you want to find an element that does not match a

condition, `std::find_if_not` is used the same way. All three functions return an iterator to the found element, or an iterator to the end of the collection if no match is found. If there are multiple matches, only the first match's iterator is returned.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> fruits = {
        "apple", "banana", "cherry", "dragonfruit", "elderberry",
        "fig", "grape", "huckleberry", "icaco", "juniper",
        "kiwi", "lemon", "melon", "nectarine", "orange", "peach",
        "quandong", "raspberry", "strawberry", "tangerine", "ugli",
        "vanilla", "watermelon", "ximenia", "yuzu", "zucchini"
    };

    auto it = std::find(fruits.begin(), fruits.end(), "blackberry");
    if (it != fruits.end()) {
        std::cout << "Found blackberry\n";
    }
    else {
        // this will print
        std::cout << "Blackberry not found\n";
    }

    it = std::find_if(fruits.begin(), fruits.end(),
        [](const std::string& fruit) {
            if (fruit[0] == 'b') return true;
            else return false;
        });

    *it = "blackberry";
    it = std::find(fruits.begin(), fruits.end(), "blackberry");
    if (it != fruits.end()) {
        // this will print
        std::cout << "Found blackberry\n";
    }
    else {
        std::cout << "Blackberry not found\n";
    }

    // print all fruits with four or fewer letters
    auto detectShortString = [](const std::string& fruit) {
        if (fruit.length() > 4) return true;
        else return false;
    };
};
```

```

it = std::find_if_not(fruits.begin(), fruits.end(), detectShortStr

while (it != fruits.end()) {
    std::cout << *it << "\n";
    ++it;
    // starting search at next element
    it = std::find_if_not(it, fruits.end(), detectShortString);
}
}

```

Find First Of

The `std::find_first_of` is similar to `std::string::find_first_of`. Its parameters are the start and end iterators of the collection to search, along with the start and end iterators of the collection of search targets. If any of the elements in the second collection are found in the first, an iterator to that element is returned.

```

#include <algorithm>
#include <array>
#include <iostream>
#include <stdlib.h>
#include <time.h>

int main() {
    srand(time(0));

    std::array<int, 100> numbers;
    std::array primeNumbers = {
        2, 3, 5, 7, 11, 13, 17, 19,
        23, 29, 31, 37, 41, 43, 47,
        53, 59, 61, 67, 71, 73, 79,
        83, 89, 97
    };

    for (int i = 0; i < 100; ++i) {
        numbers[i] = rand() % 100 + 1;
    }

    auto it = std::find_first_of(
        numbers.begin(),
        numbers.end(),
        primeNumbers.begin(),
        primeNumbers.end()
    );

    if (it != numbers.end())

```

```

        std::cout << "The first prime number found is: " << *it << "\n";
    }
    else
    {
        std::cout << "No prime numbers found in range\n";
    }

    return 0;
}

```

Count and Count If

To determine how many times an element appears in a collection, `std::count` can be used. `std::count` expects a starting iterator, ending iterator, and an element to search for and count as parameters. It returns the number of instances of the provided element in the collection as a `size_t` value. To count the number of elements that match a certain criterion, use `std::count_if`. Again, `std::count_if` expects the start and end iterator, but instead of a specific element it counts, it takes a unary predicate, and will count all elements for which that predicate returns true.

```

#include <iostream>
#include <vector>
#include <algorithm> // Required for std::count and std::count_if

int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 4, 5, 6, 4, 7, 8, 9, 10};

    int target = 4;
    // this version searches for a certain element
    auto targetCount = std::count(numbers.begin(), numbers.end(), target);
    std::cout << "The number " << target
              << " appears " << targetCount << " times.\n";

    // this version takes a unary predicate, and counts any elements that
    // return true from the predicate
    auto evenCount = std::count_if(numbers.begin(), numbers.end(),
    [](int n) {
        return n % 2 == 0;
    });
    std::cout << "There are " << evenCount << " even numbers in the list\n";

    // the predicate can be anything
    auto greaterThanFive = std::count_if(numbers.begin(), numbers.end(),
    [](int n) {
        return n > 5;
    });
    std::cout << "There are " << greaterThanFive << " numbers greater than 5\n";
}

```

```
    return 0;
}
```

Search

The `std::search` function takes two ranges as input. It searches for the first occurrence of the second range within the first, and returns an iterator to the result of the search. For example:

```
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> values = { 1, 2, 3, 1, 2, 6, 3, 1, 2 };
    std::vector<int> target = { 1, 2, 6 };
    // pass the beginning and end of the range to search
    // and the beginning and end of the range we want to find
    auto it = std::search(values.begin(), values.end(), target.begin(), target.end());
    // the iterator returned should point to the 4th element of the vector
    assert(*it == values[3]);
    assert(*(it + 1) == values[4]);
    assert(*(it + 2) == values[5]);
    target = { 1, 2, 5 };
    // since target is not in the range we are searching, we expect no hit
    it = std::search(values.begin(), values.end(), target.begin(), target.end());
    // when the target is not found it returns the second argument,
    // i.e., the end of the search range
    assert(it == values.end());
    return 0;
}
```

Search N

Rather than taking a range to search for, `search_n` looks for consecutive occurrences of a single value. The parameters are the start and end iterators of a collection to search in, the number of times the target should appear consecutively, and the target to search for.

Example:

```
#include <algorithm>
#include <cassert>
#include <vector>

int main() {
    std::vector<int> values = { 1, 1, 2, 1, 1, 1, 2, 3, 1, 1, 1, 1, 1, 1 };
    // find first occurrence of 2 1s in a row
}
```

```

    auto it2 = std::search_n(values.begin(), values.end(), 2, 1);
    // find first occurrence of 3 1s in a row
    auto it3 = std::search_n(values.begin(), values.end(), 3, 1);
    // find first occurrence of 4 1s in a row
    auto it4 = std::search_n(values.begin(), values.end(), 4, 1);
    // find first occurrence of 5 1s in a row
    auto it5 = std::search_n(values.begin(), values.end(), 5, 1);
    // find first occurrence of 6 1s in a row
    auto it6 = std::search_n(values.begin(), values.end(), 6, 1);

    assert(it2 == values.begin());
    assert(it3 == values.begin() + 3);
    assert(it4 == values.begin() + 8);
    assert(it5 == values.begin() + 8);
    assert(it6 == values.end());
    return 0;
}

```

Conclusion

This week, we looked at the first set of STL Algorithms.

`std::for_each` is used to apply a unary function to every element in a container. `std::any_of`, `std::all_of`, and `std::none_of` return `bool`, and just determine if an element does or does not appear in a collection. `std::find` looks for occurrences of a particular element, while `std::find_if` searches for elements that cause a unary predicate to return true. The `std::count` and `std::count_if` functions return the number of times an element (or elements that match a predicate) occur in a collection. The `std::search` function takes two ranges as input, and searches for the first occurrence of a range within another range. The `std::search_n` function searches a range for consecutive occurrences of elements.